Suggested solutions to Compulsory Assignment in BAN404, spring 2025x|

Task 1

Describe relevant features of the output variable with descriptive statistics (tables and graphs). Also use descriptive statistics to investigate promising predictors for lsalary. Remove variables that cannot possibly be available when the salary of a new CEO should be predicted.

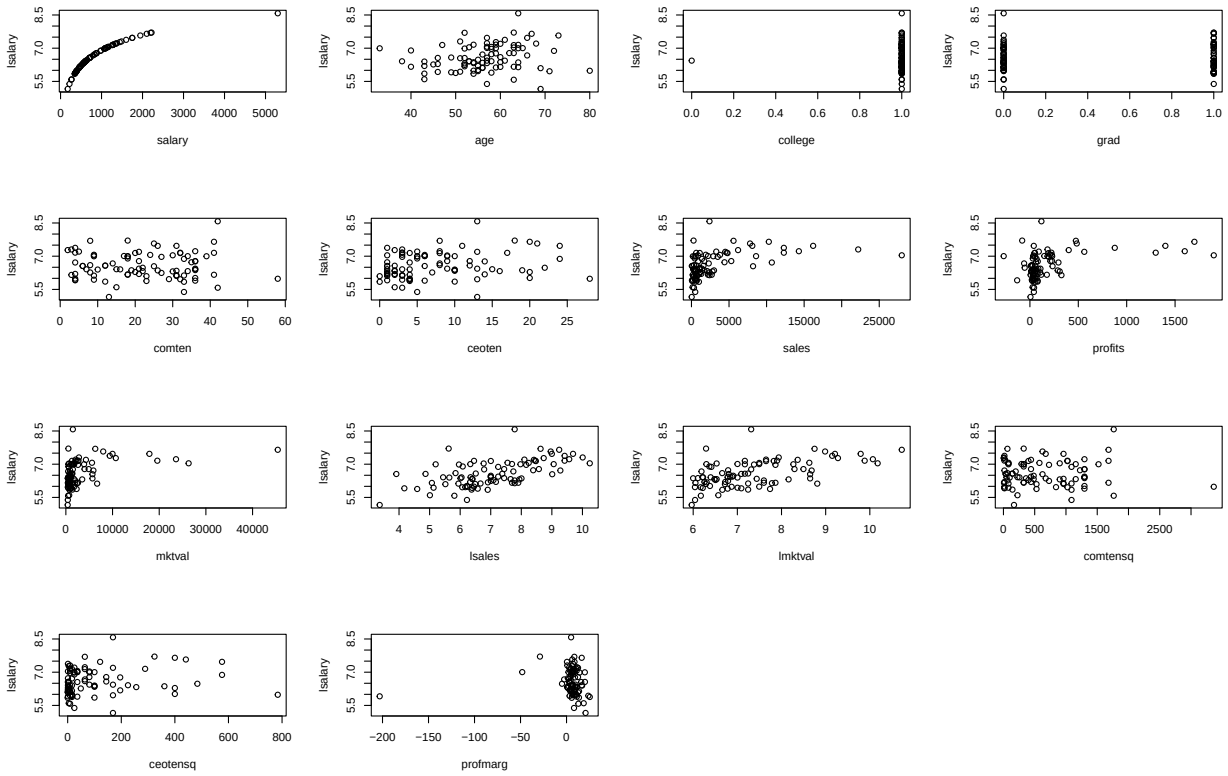
Suggested solution

First, the data is splitted into training and test data. I split it 50/50.

```
library(wooldridge)
set.seed(1234)
n <- nrow(ceosal2)
ntrain <- floor(n/2)
ind <- sample(1:n,size=ntrain)
train <- ceosal2[ind,]
test <- ceosal2[-ind,]
```

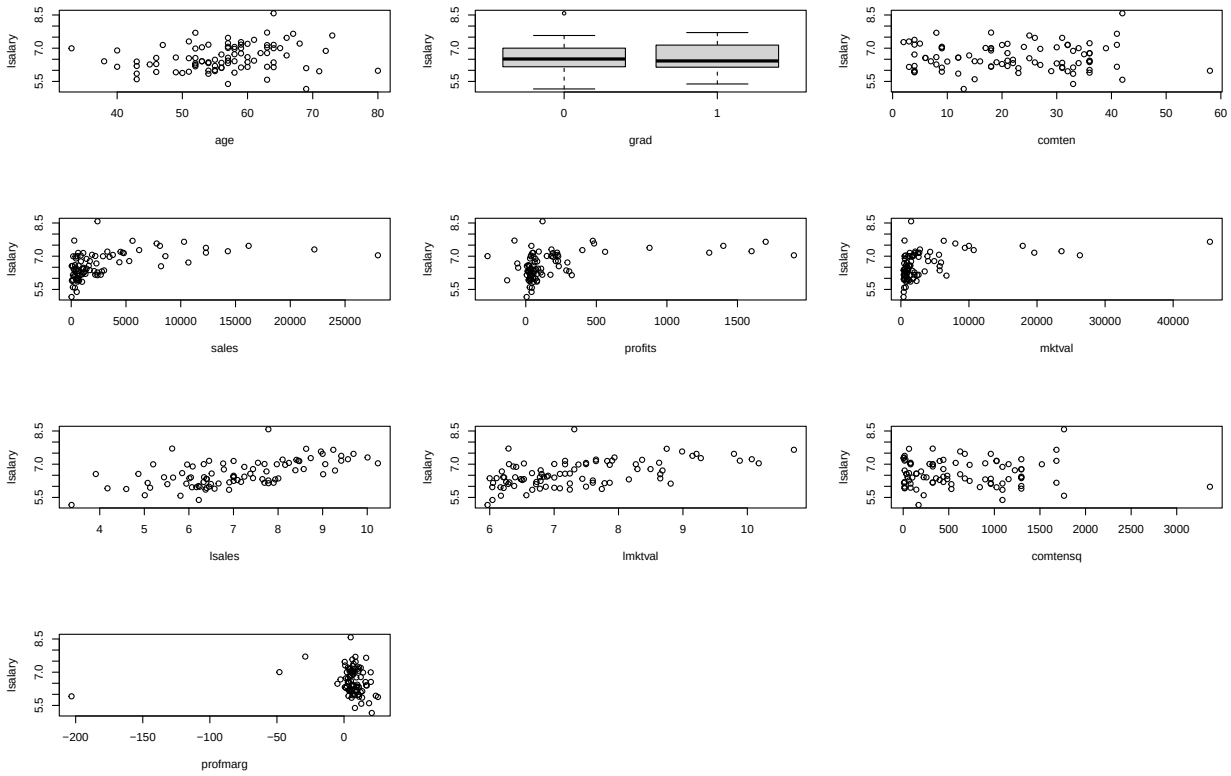
Now, I plot `lsalary` against all the other variables

```
par(mfrow=c(4,4))
plot(lsalary~.,data=train)
```



`salary` is not available at the time when a new CEO is to be hired. This is simply a deterministic transformation of the variable that should be predicted. I assume that the new CEO is to be hired from inside the company. That means that `comten` and its square is available but not `ceoten` and its square. I assume that all other variables are available and use them as potential predictors. Another premise I set is that the purpose is to predict the new CEO's (logarithm of) salary, not to set a fair salary. Also, there are only one of the CEOs in the training data which did not attend college and I therefore remove this variable since I do not think that it will be very useful. I now remove the predictors not available at the time of a prediction and plot `lsalary` against them. A categorical variable is converted to the `factor` type.

```
ceosal3 <- ceosal2[,!(names(ceosal2) %in% c("college","salary","ceoten","ceotensq"))]
ceosal3$grad <- as.factor(ceosal3$grad)
train <- ceosal3[ind,]
test <- ceosal3[-ind,]
par(mfrow=c(4,3))
plot(lsalary~.,data=train)
```



There are not a lot of obviously strong predictors for `lsalary` in the dataset. The logarithm of sales, `lsales`, is perhaps the one that strikes the eye most immediately. Note that `sales` is still in the dataset. It obviously contains the same information as `lsales` but could potentially be useful if the relationship is non-linear. I therefore keep it. For the categorical variable `grad` 95% confidence intervals of `lsalary` are computed for each category.

```
by(train$lsalary,train$grad,t.test)
```

```
## train$grad: 0
##
##  One Sample t-test
##
## data:  dd[x, ]
## t = 75.036, df = 45, p-value < 2.2e-16
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  6.405181 6.758519
## sample estimates:
## mean of x
##   6.58185
##
## ------------------------------------------------------------
## train$grad: 1
##
##  One Sample t-test
##
## data:  dd[x, ]
## t = 71.408, df = 41, p-value < 2.2e-16
```

```
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  6.399417 6.771924
## sample estimates:
## mean of x
##   6.58567
```

As can bee seen from the output, the confidence intervals are very similar, and I see no reason to believe that `grad` is a promising predictor.

To summarize, `lsales` is the only obviously promising predictor. I will however investigate the others as well in Task 2.

Task 2

Use linear regression (on all or a subset of the predictors) and LASSO to predict lsalary. Evaluate the predictions on test data using an appropriate error measure.

Suggested solution

First I use linear regression with all variables. Given that it is prediction of a numerical variable and I have no information on the consequences of decisions made based on the prediction, I use mean square error (MSE) to evaluate the predictions.

```
ols <- lm(lsalary~.,data=train)
pred_ols <- predict(ols,newdata=test)
mse_ols <- mean((test$lsalary-pred_ols)^2)
mse_ols
```

```
## [1] 0.3044908
```

I now investigate if a linear prediction based on only `lsales` is to prefer.

```
ols_lsales <- lm(lsalary~lsales,data=train)
pred_lsales <- predict(ols_lsales,newdata=test)
mse_lsales <- mean((test$lsalary-pred_lsales)^2)
mse_lsales
```

```
## [1] 0.309084
```

The difference is very small. Comparing adjusted R-squares from the estimated models give the same conclusion. The adjusted R-squares are very similar. However, experimenting with different seeds, larger difference is sometimes observed.

Leave-one-out (LOO) cross-validation is, therefore, implemented

```
SE_ols <- matrix(0,n,1)
SE_lsales <- SE_ols
for(i in 1:n)
{
  train <- ceosal3[-i,]
  test <- ceosal3[i,]
  ols <- lm(lsalary~.,data=train)
  pred_ols <- predict(ols,newdata=test)
  SE_ols[i] <- (test$lsalary-pred_ols)^2
  ols_lsales <- lm(lsalary~lsales,data=train)
  pred_lsales <- predict(ols_lsales,newdata=test)
  SE_lsales[i] <- (test$lsalary-pred_lsales)^2
}
mse_ols <- mean(SE_ols)
```

```
mse_lsales <- mean(SE_lsales)
mse_ols
```

```
## [1] 0.4253121
```

```
mse_lsales
```

```
## [1] 0.2685442
```

This more realistic (since we use 176 observations to fit the model; in reality we would have had 177) evaluation shows that the model with only `lsales` is clearly better. It is also ok to continue with validation set cross-validation. However, it is then important to investigate results with different seeds.

I now use the LASSO with all variables, mainly to investigate if it manages to identify `lsales`. First, this is done by fitting the model on the training data made in the beginning of the task. This is in order to investigate the coefficents and see which ones are made equal to zero by LASSO. Cross validation to determine `lambda` in the LASSO-estimation is made with the `cv.glmnet`-function.

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-8
```

```
train <- ceosal3[ind,]
test <-  ceosal3[-ind,]
Xtrain <- train[,-7]
ytrain <- train[,7]
la <- cv.glmnet(as.matrix(Xtrain),ytrain,alpha=1)$lambda.min
lasso <- glmnet(as.matrix(Xtrain),ytrain,alpha=1,lambda=la)
cbind(coef(ols),coef(lasso))
```

```
## 11 x 2 sparse Matrix of class "dgCMatrix"
##                                  s0
## (Intercept)  4.642819e+00 5.3144588
## age          5.592014e-03 .
## grad1       -1.144456e-01 .
## comten       8.378669e-04 .
## sales       -1.008487e-05 .
## profits      3.792347e-05 .
## mktval       1.313392e-05 .
## lsales       1.938689e-01 0.1204753
## lmktval      5.522672e-02 0.0563756
## comtensq    -2.245535e-04 .
## profmarg    -2.305640e-03 .
```

Only `lsales` and `lmktval` are choosen by LASSO. I now use LOOCV to evaluate the predictions.

```
SE_lasso <- matrix(0,n,1)
for(i in 1:n)
{
  Xtrain <- ceosal3[-i,-7]
  ytrain <- ceosal3[-i,7]
  Xtest <- ceosal3[i,-7]
  ytest <- ceosal3[i,7]
  cv_lasso <- cv.glmnet(as.matrix(Xtrain),ytrain,alpha=1)
  lasso <- glmnet(Xtrain,ytrain,data=train,alpha=1,
                  lambda=cv_lasso$lambda.min)
```

```
  pred_lasso <- predict(lasso,newx=Xtest)
  SE_lasso[i] <- (ytest-pred_lasso)^2
}
mse_lasso <- mean(SE_lasso)
mse_lasso
```

```
## [1] 0.3777567
```

This works better than OLS but worse than the model with `lsales` only.

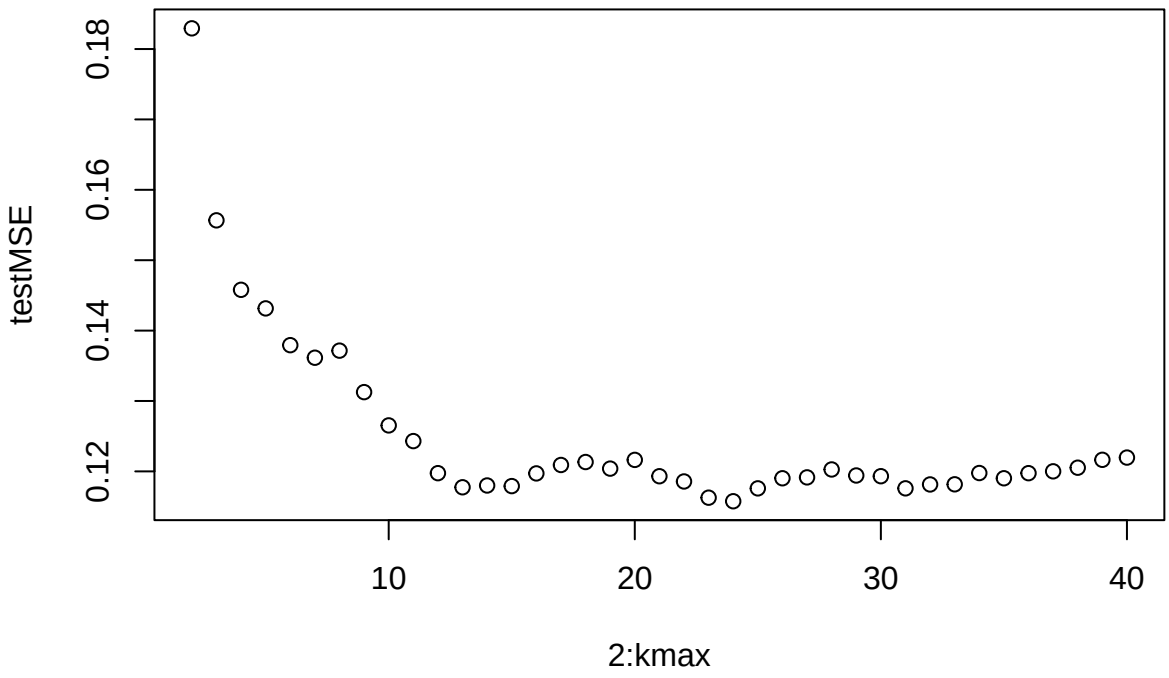
Task 3

Predict lsalary by by KNN, using only textcolor{blue}{lsales} as a predictor. Determine K using leave-one-out cross-validation. Also, evaluate the predictions and compare with the results in 2.

Suggested solution

If LOOCV is used for both determining K and for evaluating the predictions, the K should, if one should be strict, be determined each time a new prediction is computed. I take a less strict approach here and determine K once based on the training data created in Task 1. Thus, be aware that some data from the test data in the evaluation of the predictions have been used in determining K. First, determine K

```
knn=function(x0,x,y,K=20)
{
d=abs(x-x0)
o=order(d)[1:K]
ypred=mean(y[o])
return(ypred)
}
train <- ceosal3[ind,]
SE <- matrix(0,n,1)
kmax <- 40
testMSE <- matrix(0,kmax-1,1)
for(kval in 2:kmax)
{
  for(i in 1:ntrain)
  {
    Xtrain <- train$lsales[-i]
    ytrain <- train$lsalary[-i]
    Xtest <- train$lsales[i]
    ytest <- train$lsalary[i]
    pred <- knn(Xtest,Xtrain,ytrain,K=kval)
    SE[i] <- (ytest-pred)^2
  }
  testMSE[kval-1] <- mean(SE)
}
plot(2:kmax,testMSE)
```




```
which.min(testMSE)
```

```
## [1] 23
```

The optimal K is, thus, 24 (note that we start investigating at K=2)

Now we evaluate the predictions using $K = 24$

```
SE_KNN <- matrix(0,n,1)
for(i in 1:n)
{
  Xtrain <- ceosal3$lsales[-i]
  ytrain <- ceosal3$lsalary[-i]
  Xtest <- ceosal3$lsales[i]
  ytest <- ceosal3$lsalary[i]
  pred_KNN <- knn(Xtest,Xtrain,ytrain,K=24)
  SE_KNN[i] <- (ytest-pred_KNN)^2
}
mse_KNN <- mean(SE_KNN)
mse_KNN
```

```
## [1] 0.2840139
```

Slightly worse than the simple linear regression with `lsales` as the predictor.

Task 4

Explain how you would use KNN if you had more than one predictor. Two alternative ways to do this have been mentioned in the course. If you would like to you can also write R-code to do this but this is not

required to pass the assignment.

Suggested solution

Two ways are

1. To compute the distance between `x0` and the observed x-values in a different way, e.g., by using a Euclidean distance. Example of distance between the test observation (x_{10}, x_{20}) and observations i, (x_{1i}, x_{2i}) for the case with two predictors

$$d_i = \sqrt{(x_{10} - x_{1i})^2 + (x_{20} - x_{2i})^2}$$

2. In the generalized additive model (GAM) framework KNN can be used as the univariate method used sequentially for all predictors. See, e.g., slides to lecture 10 where a a generalization of KNN, the Nadaraya-Watson estimator is used as an example.

Task 4

Argue for which, if any, variables might have a nonlinear relationship with lsalary and find a good model within the generalized additive models (GAM) framework. Evaluate the predictions.

Suggested solution

In the univariate plots produced for the answer to Task 1 `sales` and `mktval` seem to have non-linear relationship to `lsalary`. On the other hand, the logarithms of those variables is also included in the dataset as predictors and looks linearly related to `lsalary`. I will nevertheless investigate these two "non-logarithmed" as predictors in their own right. I investigate model visually based on the training data from Task 1 but evaluate the predictions by LOOCV.

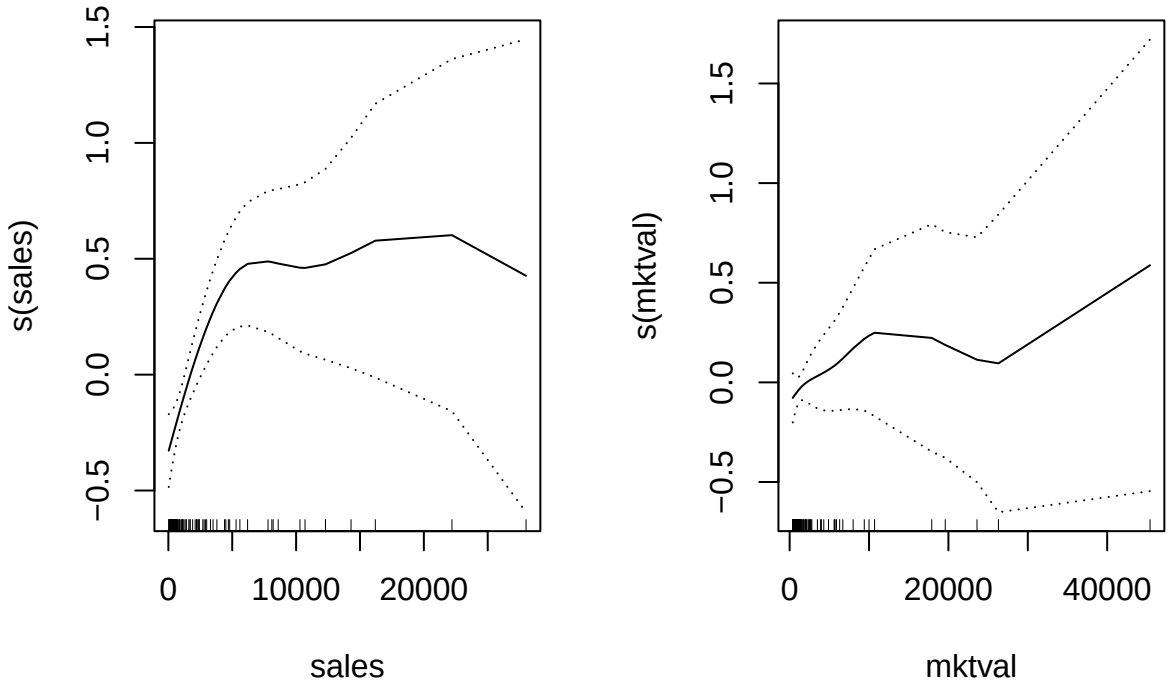
```
library(gam)
```

```
## Loading required package: splines
```

```
## Loading required package: foreach
```

```
## Loaded gam 1.22-5
```

```
train <- ceosal3[ind,]
test <-  ceosal3[-ind,]
gam1 <- gam(lsalary ~ s(sales) + s(mktval),data=train)
par(mfrow=c(1,2))
plot(gam1,se=TRUE,terms=c("s(sales)","s(mktval"))
```



The contribution of the non-linear `mktval` looks questionable. I therefore only continue with the `s(sales)`-term.

```
SE_ <- matrix(0,n,1)
for(i in 1:n)
{
  train <- ceosal3[-i,]
  test <- ceosal3[i,]
  gam1 <- gam(lsalary ~ s(sales), data=train)
  pred_sales <- predict(gam1,newdata=test)
  SE[i] <- (test$lsalary-pred_sales)^2
}
mse_sales <- mean(SE)
mse_sales
```

```
## [1] 0.2918272
```

Slightly worse than KNN with `lsales` as predictor.